

Tóm tắt toàn dự án: Sentiment tin tức chứng khoán Việt Nam và VN-Index

Table of contents

1	Mục tiêu dự án	2
2	Dữ liệu dùng chung	2
2.1	Chất lượng ground truth — trần liên chủ quan (inter-annotator agreement)	2
3	Kết quả từng phương pháp	3
3.1	1. Lexicon-based	3
3.1.1	Luồng làm việc	3
3.1.2	Kiểm định độ chính xác của việc triển khai	4
3.2	2. Traditional ML	5
3.2.1	Luồng làm việc	5
3.2.2	Kiểm định độ chính xác của việc triển khai	7
3.2.3	Quyết định lại hyperparameter $C=0,1$ của LogReg/SVM — đã kiểm định lại, kết quả MÃU THUẦN	8
3.3	3. Transfer Learning (PhoBERT)	9
3.3.1	Luồng làm việc	9
3.3.2	Kiểm định độ chính xác của việc triển khai	10
4	So sánh 3 nhánh trên cùng Holdout (314 bài, kiểm tra một lần)	11
5	Câu hỏi chính: sentiment tin tức có dự báo được VN-Index không?	11
5.1	Kết luận: KHÔNG — null vững qua mọi kiểm định	11
6	Phát hiện phụ đáng chú ý: khối lượng giao dịch khối nội	12
6.0.1	Vì sao PhoBERT và Lexicon cho kết quả khác nhau — đã điều tra trực tiếp trên dữ liệu	13
7	Kết luận chung toàn dự án	14
7.1	Việc còn mở	14

1 Mục tiêu dự án

Dự án đo **sentiment** (cảm xúc: negative/neutral/positive) của tin tức chứng khoán Việt Nam ở cấp bài báo, rồi kiểm định xem sentiment tổng hợp theo ngày có **dự báo được lợi suất và khối lượng giao dịch của VN-Index** hay không. Việc gán nhãn sentiment được làm theo **ba phương pháp độc lập**, để vừa so sánh chất lượng phân loại, vừa kiểm tra xem kết luận về VN-Index có phụ thuộc vào việc chọn phương pháp đo hay không:

1. **Lexicon-based** — từ điển tài chính (Loughran & McDonald) + công thức tính điểm, không cần huấn luyện.
2. **Traditional ML** — TF-IDF + các classifier cổ điển (Logistic Regression, Naive Bayes, SVM, Random Forest).
3. **Transfer Learning** — fine-tune PhoBERT (mô hình ngôn ngữ tiếng Việt đã pretrain).

2 Dữ liệu dùng chung

Thành phần	Số lượng
Ground truth gán nhãn tay	1.044 bài
Phân bố nhãn	negative 296 (28,4%) / neutral 512 (49,0%) / positive 236 (22,6%)
Kho tin chưa gán nhãn	126.576 bài
Tách Tune / Holdout (cố định, dùng chung cả 3 nhánh)	730 / 314

Cả 3 nhánh dùng **chung đúng 1 bộ Tune/Holdout** — điều kiện tiên quyết để so sánh công bằng giữa các phương pháp.

2.1 Chất lượng ground truth — trần liên chủ quan (inter-annotator agreement)

Đo trên 416 bài được **2 người gán độc lập, mù, xáo thứ tự** (annotator 1 = ground truth gốc, annotator 2 = người thứ hai):

Chỉ số	Giá trị
Raw agreement (khớp nguyên văn)	74,28%
Weighted kappa (quadratic — 3 lớp có thứ tự)	0,7134
Krippendorff's ordinal alpha (đối chiếu độc lập)	0,7164

2 phương pháp thống kê độc lập (kappa và Krippendorff's alpha) cho kết quả gần như trùng khít — độ đồng thuận thật giữa 2 người là mức “khá tốt” (substantial, Landis–Koch) nhưng vẫn còn ~28-29% không giải thích được bởi đồng thuận ngẫu nhiên. Đây là **trần tham chiếu**: một model không nên kỳ vọng khớp nhãn người tốt hơn hẳn mức người-với-người tự đồng ý với nhau. Model tốt nhất của dự án (PhoBERT E3) đạt 77,6-79,3% khớp với 2 annotator trên đúng 416 bài này —

vượt trần raw agreement — hiện tượng đã biết trong tài liệu (model học một hàm quyết định ổn định/có hệ thống, trong khi mỗi người mang nhiều ngẫu nhiên cá nhân không tương quan với nhau).

3 Kết quả từng phương pháp

3.1 1. Lexicon-based

3.1.1 Luồng làm việc

1. **Xây từ điển hạt giống (seed dictionary):** 7 category kiểu Loughran & McDonald (2011) cho văn bản tài chính — negative, positive, uncertainty, litigious, strong_modal, weak_modal, constraining — dịch/khởi tạo thủ công cho tiếng Việt (Seed_set_Prepare/final_seed/).
2. **Mở rộng từ điển bằng “bootstrap” bán giám sát, 4 vòng lặp** (lưu ý: đây là nghĩa “tự mở rộng có người duyệt lặp lại”, khác hẳn “bootstrap” thống kê — resampling để tính CI — dùng ở phần Traditional ML/Transfer Learning phía sau). Mỗi vòng (build_sentiment_dictionary_pmi_bootstrap.py) lặp lại đúng 1 chu trình:

- a. Tính **PMI** giữa mọi candidate n-gram (2-3 từ, xuất hiện ≥ 20 tài liệu, quét trên 126.576 bài chưa nhãn) với **seed hiện tại** của từng category. Đơn vị đồng xuất hiện là **câu** (không phải toàn bài) — 2 từ được tính “đi cùng nhau” nếu nằm chung 1 câu, cửa sổ hẹp hơn giúp PMI phản ánh đúng quan hệ ngữ nghĩa thay vì chỉ “cùng chủ đề bài viết”. Công thức có làm trơn Laplace (add- α , $\alpha = 1$) để không bao giờ ra NaN khi 1 cặp chưa từng gặp nhau:

$$\text{PMI}(t, s) = \log_2 \frac{(\text{co_df}_{t,s} + \alpha) \cdot N}{(\text{df}_t + \alpha) \cdot (\text{df}_s + \alpha)}$$

Mỗi seed viết tay trước tiên được **ánh xạ về đúng dạng thực tế** xuất hiện trong corpus đã tokenize (1 seed có thể ứng với nhiều biến thể tách từ), loại các seed quá hiếm ($\text{df} < 20$) hoặc quá phổ biến (df/N vượt ngưỡng, ít giá trị phân biệt) khỏi bước tính PMI. Điểm PMI thô sau đó được chuẩn hoá kiểu two-way (trừ trung bình theo hàng/theo cột/toàn cục, giống phần dư ANOVA 2 chiều) để bớt thiên lệch do n-gram tần suất cao chung chung được mọi category cùng chấm điểm cao.

- b. Lấy **top 3 percentile** điểm PMI mỗi category (ứng viên xuất hiện đủ nhiều, candidate_unit_st 50), xuất ra review_round{N}.csv — **người review duyệt tay từng term** (cột approve), không tự động đưa thẳng vào từ điển chỉ vì điểm PMI cao.
- c. Term được duyệt nhập vào seed của vòng kế tiếp (seed_round{N}/), đồng thời ghi log gốc gác đầy đủ vào seed_provenance.csv (term nào, vòng nào, điểm gì lúc duyệt, ngày nào) — truy vết lại được toàn bộ lịch sử mở rộng của từng term.
- d. Lặp lại bước (a) với seed đã mở rộng; tổng cộng 4 vòng (round 1→4), một số category được bỏ qua ở vòng sau khi ứng viên mới đã bão hoà (--skip).

- **Cách 1 (PMI):** dùng đúng cơ chế bootstrap trên cho cả 7 category.

- **Cách 2 (Intensity)**: hệ số cường độ nội tại kiểu VADER, tự thiết kế trên cùng tập seed đã mở rộng (không chạy lại vòng bootstrap PMI riêng; không phải do chuyên gia chấm độc lập từng từ như VADER gốc).
 - Cả 2 gộp thành `weighted_dictionary.csv` (1.767 term): `final_seed` (seed gán tay ban đầu) = trọng số 1,0; term thêm qua bootstrap = `score_centered` tại đúng vòng nó được duyệt.
3. **Tính điểm từng bài** (`score_articles.py`): với mỗi category, quét khớp n-gram trong từ điển lên văn bản đã tokenize, mỗi lần khớp được gán dấu:

$$S_{cat} = \frac{\sum_i \text{weight}(w_i) \cdot \text{sign}(w_i)}{N_{total}}$$

- `sign(w_i)` = -1 nếu quét ngược tới đa 4 token trước điểm khớp (cùng câu) gặp từ phủ định (`negation_cue_words.txt`) **trước khi** gặp từ nổi tương phản như “nhưng/tuy nhiên” (từ nổi là điểm dừng cứng, chặn phủ định lan sang mệnh đề sau); ngược lại +1.
 - `N_total` = tổng số token của bài, để bài dài/ngắn không lệch điểm vì độ dài.
4. **Quyết định nhãn 3 lớp**: `net_sentiment_score = positive_score - negative_score`, so với một **ngưỡng trung lập (neutral margin)** được hiệu chỉnh thực nghiệm trên tập Tune (`tune_neutral_margin.py`), không chọn tùy ý.
5. **Cách khớp n-gram**: hiện dùng longest-match (ưu tiên cụm dài nhất khi có chồng lấn); `NEGATION_WINDOW=4` cũng là giá trị mặc định.

	Holdout (314 bài)
PMI — accuracy / macro-F1	66,2% / 0,650
Intensity — accuracy / macro-F1	67,2% / 0,662

Không cần huấn luyện, nhanh, dễ diễn giải — nhưng độ chính xác thấp nhất trong 3 phương pháp.

3.1.2 Kiểm định độ chính xác của việc triển khai

- **Bootstrap mở rộng từ điển có người duyệt + log truy vết**: mỗi term mới không được đưa thẳng vào từ điển chỉ vì điểm PMI cao — phải qua bước người review duyệt tay (`review_round{N}.csv`), và mọi term được duyệt đều ghi lại vòng/điểm/ngày vào `seed_provenance.csv` — nếu một term hoá ra sai (VD nhiều do PMI cao ngẫu nhiên), có thể truy ngược đúng vòng nó được thêm vào để sửa, không phải dò lại toàn bộ 1.767 term.
- **Chặn artifact PMI giả do quan hệ “chứa nhau”**: nếu 1 seed (VD “giảm”) nằm lọt bên trong 1 candidate n-gram (VD “giảm lỗ”), 2 chuỗi này gần như luôn xuất hiện cùng câu chỉ vì chúng thực chất là cùng 1 chỗ trong văn bản, không phải vì có quan hệ ngữ nghĩa — dễ đẩy PMI lên cao giả tạo. Code loại riêng từng cặp (candidate, seed) bị lồng nhau kiểu này khỏi phép tính, thay vì loại bỏ nhầm cả candidate hoặc cả seed.
- **Hiệu chỉnh tham số bằng dữ liệu, không chọn tùy ý**: cửa sổ phủ định và ngưỡng trung lập đều được quét qua nhiều giá trị trên tập Tune (`tune_negation_window.py`, `tune_neutral_margin.py`) rồi mới chốt, thay vì gán số cố định theo cảm tính.

- **So sánh chiến lược khớp** (`compare_longest_vs_overlapping_match.py`): phát hiện khớp chồng lấn (`overlapping-match`) thắng rõ `longest-match` ở cả Tune lẫn Holdout, cả 2 cách tính trọng số (chênh ~2-3 điểm % accuracy) — cho thấy cách khớp hiện tại trong production **chưa tối ưu**; đây là một khoảng cách triển khai đã đo được nhưng chưa sửa (xem “Việc còn mở”).
- **Truy nguyên lỗi khớp từ bằng dữ liệu thật, không suy đoán** (`diagnose_missing_terms.py`, vừa chạy lại 2026-09-27 trên từ điển + ground truth 1.044 dòng hiện tại — script này trước đó chưa từng được chạy, số “~44%”/“35-45%” trong docstring cũ là ước lượng cũ trên ground truth nhỏ hơn, **không dùng nữa**):
 - (a) **N-gram “chết”**: 141/1.733 term (8,1%) không match được lần nào trên toàn bộ 126.576 bài. Gần như toàn bộ là cụm 3-6 từ mà VNCoreNLP **đã tự gộp 2 âm tiết cuối thành 1 token có gạch dưới**, khiến cụm từ điển (viết rời từng âm tiết) không còn khớp được với chuỗi token thật — ví dụ cụ thể, kiểm chứng trực tiếp trên corpus:
 - * Từ điển có “vượt kế hoạch” (3 token rời: vượt / kế / hoạch) — nhưng câu thật luôn tokenize thành “... vượt so với kế_hoạch năm”: “kế_hoạch” đã là **1 token duy nhất** (xuất hiện 14.490 lần chỉ trong 30.000 bài đầu) → cụm 3-token của từ điển không bao giờ khớp được với thực tế chỉ có 2 token.
 - * Tương tự “tái cấu trúc” (từ điển: 3 token) vs câu thật “... đã thực_hiện tái_cấu_trúc tại đơn_vị ...” (cấu_trúc gộp thành 1 token, 1.367 lần/30.000 bài).
 - * Và “vượt kỳ vọng” vs kỳ_vọng (gộp 1 token, 2.313 lần/30.000 bài) — cùng một lỗi hệ thống, không phải case đơn lẻ.
 - (b) **Từ thiếu**: chỉ **85/1.044 bài (8,1%)** ground truth bị chấm “Neutral_no_match” nhưng nhãn thật Positive (44 bài) hoặc Negative (41 bài) — thấp hơn nhiều so với số cũ. Từ xuất hiện nhiều nhất trong nhóm này lại là từ vựng báo cáo tài chính chung chung (tỷ đồng, tăng, doanh_thu, lợi_nhuận, cổ_tức cho Positive; giảm, hđqt, chứng_khoán cho Negative) — **chưa thể thêm thẳng vào từ điển**: bản thân “tăng”/“giảm” không mang cực tính cố định (tăng *chi phí* là tin xấu, tăng *lợi nhuận* là tin tốt) nên thêm nhầm sẽ gây false positive ở nơi khác; đây là giới hạn thật của cách tiếp cận “túi từ” (bag-of-terms), không phải chỉ do từ điển thiếu từ.
- **Kiểm tra hợp lý bề mặt (face validity)**: 5 bài có `net_sentiment_score` cao nhất trên toàn corpus đều là tin lợi nhuận/tăng trưởng vượt kế hoạch — khớp trực giác, dấu hiệu công thức tính điểm hoạt động đúng hướng trước khi đối chiếu số liệu định lượng.

3.2 2. Traditional ML

3.2.1 Luồng làm việc

1. **Đặc trưng TF-IDF** (n-gram 1-3 trên token tiếng Việt): lọc từ vựng theo `min_df` tỷ lệ (2% cho unigram, 1,3% cho bi/trigram) và `max_df=85%`, bỏ stopwords/số/token lỗi → 1.001 term trên 1.044 bài (583 unigram / 293 bigram / 125 trigram). Trọng số dùng đúng công thức Loughran & McDonald (2011) cho văn bản tài chính (biến thể của TF-IDF, chuẩn hoá theo độ dài bài bằng $1 + \log(a_j)$ ở mẫu số). Từ vựng, `df`, `idf` và bước chọn top **5.000 feature** đều được **fit lại riêng trong từng train-fold** — không fit một lần trên toàn bộ dữ liệu rồi mới chia fold.

2. **Đặc trưng lexicon bổ sung** (chỉ Random Forest): 9 cột — 7 tỷ lệ khớp từ điển tài chính theo category (hits/total_tokenizer), net_polarity = positive_prop - negative_prop, coverage = positive_prop + negative_prop — ghép cạnh ma trận TF-IDF, chuẩn hoá z-score trong từng fold. Vì từ điển và luật phủ định đều **cố định từ trước** (không fit từ ground truth), tính đặc trưng này trên fold validation/Holdout không rò rỉ gì. 2 điểm kỹ thuật đáng chú ý:
 - **Khớp cụm nhiều token**: nhiều entry trong từ điển dài 3-6 âm tiết trong khi so khớp đơn-token bỏ sót phần lớn (đo trên tập tune: chỉ 5,6-37,5% mỗi category từng khớp dạng 1 token) — nên code dựng mọi n-gram nối _ của dòng token tới đúng độ dài cụm dài nhất **tính động từ chính các danh sách từ đang dùng** (không hardcoded), rồi mới so khớp.
 - **Cố ý làm khác Loughran & McDonald (2011)**: bản gốc chỉ phủ định nhóm positive (lý do L&M đưa ra: “not terrible earnings” không xảy ra trong văn bản tài chính nên phủ định nhóm negative là thừa). Ở đây áp **cùng một quy tắc phủ định cho cả positive lẫn negative** (cửa sổ 3 token) — một phép kiểm chứng thực nghiệm giả định đó có đúng với tin tức tiếng Việt hay không, thay vì mặc nhiên tin theo.
3. **6 model production**, mỗi model có 1 lựa chọn kỹ thuật riêng cần lưu ý:
 - **Logistic Regression**: C=0,1, class_weight="balanced", max_iter=5000.
 - **Linear SVM**: LinearSVC(C=0,1, class_weight="balanced") — LinearSVC không có predict_proba gốc; “xác suất” ở đây là **softmax tự áp lên decision_function** (MarginSoftmaxSVC), không phải xác suất đã hiệu chỉnh. Từng thử Platt scaling (CalibratedClassifierCV) nhưng bỏ vì mỗi inner-fold hiệu chỉnh chỉ có ~40 dòng, quá ít để tin cậy (đo trực tiếp bằng calibration_check.py). Vì vậy SVM bị loại khỏi Ensemble.
 - **Random Forest**: 300 cây, class_weight="balanced", xác suất qua CalibratedClassifierCV (method=cv=3) — isotonic cần nhiều dữ liệu hơn Platt nhưng ít bị méo hơn khi đủ dữ liệu, đã đo trực tiếp bằng reliability diagram + Brier score ở cả quy mô 599 và 1.044 dòng trước khi chọn.
 - **Ensemble** = trung bình xác suất Logistic Regression + Multinomial NB + Random Forest (không gồm SVM, lý do trên).
 - Đầu ra mỗi bài: nhãn + xác suất 3 lớp + sentiment_score_ml = P(positive) - P(negative).
4. **Đánh giá production**: 10 lần lặp × 5-fold stratified CV, chạy **đúng một lần** trên toàn bộ 1.044 dòng; xác suất out-of-fold được lưu lại và **dùng lại** cho mọi kiểm định thống kê phía sau (McNemar, bootstrap, Nadeau-Bengio, calibration) — không train lại nhiều lần cho từng câu hỏi.
5. **Nested-CV tuning** (tune_hyperparameters.py, chỉ trên tập Tune 730 dòng): với mỗi outer fold (5 fold), tham số được chọn bằng một vòng CV nội bộ (3 fold) restricted vào phần train của outer fold đó — outer-validation chỉ bị “nhìn” đúng 1 lần, sau khi tham số đã chốt, tránh rò rỉ tinh chỉnh tham số kiểu Cawley & Talbot (2010).

Model	Macro F1 (10×5-fold CV, 1.044 bài)
Random Forest (+lexicon, isotonic)	0,708 ± 0,008
Logistic Regression	0,687 ± 0,007

Model	Macro F1 (10×5-fold CV, 1.044 bài)
Ensemble	0,678 ± 0,006
SVM	0,663 ± 0,008
Multinomial NB	0,647 ± 0,004
Complement NB	0,636 ± 0,004

Kiểm định thống kê rất kỹ (Nadeau-Bengio corrected variance, permutation leakage test, nested-CV tuning, calibration check): **{Logistic Regression, Random Forest} > {Naive Bayes}** là phát hiện vững; việc thêm đặc trưng lexicon vào Random Forest cải thiện đáng tin ($\Delta=+0,049$ trên Holdout, $p=0,003$).

3.2.2 Kiểm định độ chính xác của việc triển khai

- **Permutation/label-shuffle leakage test** (Ojala & Garriga 2010): xáo trộn nhãn (quan hệ text–nhãn = 0 theo thiết kế) rồi chạy đúng harness CV leak-free 30 lần; nếu harness thật sự leak-free, macro-F1 trên nhãn xáo trộn phải rơi về mức ngẫu nhiên. Kết quả: điểm thật (0,670 cho Naive Bayes-TF-IDF, 0,691 cho Random Forest+lexicon) vượt qua **toàn bộ 30/30** lần xáo trộn (cách trung bình shuffle 16-25 độ lệch chuẩn) — kể cả ở khối +lexicon vốn có thêm bước chuẩn hoá z-score trong fold (nguồn rò rỉ tiềm ẩn riêng) cũng không phát hiện rò rỉ.
- **Đối chiếu độc lập bằng code ngoài dự án**: pipeline `TfidfVectorizer+LogisticRegression` thuần của `sklearn` (`cross_val_predict`, không dùng bất kỳ code nào của dự án) trên cùng 730 dòng Tune cho macro-F1 0,7223 — so với $0,678 \pm 0,012$ của chính dự án trên cùng tập, lệch +0,044, còn trong ngưỡng $\pm 0,05$ dự án tự đặt để coi là “cùng tầm” nhưng khá sát cận trên (đáng theo dõi khi ground truth lớn hơn).
- **Kiểm định chất lượng hiệu chỉnh xác suất** (calibration check): so 6 biến thể trên cùng 1 cách chia fold — isotonic calibration cải thiện rõ cả Brier ($0,4022 \rightarrow 0,3860$ và $0,3807 \rightarrow 0,3682$) lẫn ECE ($0,0801 \rightarrow 0,0304$ và $0,0632 \rightarrow 0,0236$) ở cả nhánh TF-IDF thuần lẫn +lexicon, gần như không đổi macro-F1 — xác nhận bước hiệu chỉnh hoạt động đúng như thiết kế (xác suất đáng tin hơn, không phải chỉ đổi nhãn dự đoán).
- **Xác nhận đặc trưng lexicon bằng Holdout độc lập** (mục 6.2 ML_SUMMARY.qmd): hiệu ứng +lexicon trên Holdout ($\Delta=+0,049$) **lớn hơn** trên Tune ($\Delta=+0,024$) — không có dấu hiệu overfit vào tập dùng để phát triển đặc trưng.
- **Cơ chế 2 phép kiểm định thống kê chính, để mentor đối chiếu công thức**:
 - *Bootstrap trên macro-F1* (`Common/bootstrap.py`): resample **có hoàn lại** 1.044 dòng, 2.000 lần; mỗi lần resample dùng **chung đúng 1 bộ chỉ số dòng** cho mọi model đang so sánh (paired bootstrap) — để $\text{delta} = \text{model_A} - \text{model_B}$ phản ánh đúng chênh lệch trên cùng một mẫu giả lập, không lẫn nhiễu do 2 model được resample khác nhau. $p\text{-value} = 2 \times \text{tỷ lệ resample rơi về phía “sai” của 0}$.
 - *Nadeau & Bengio (2003) corrected variance*: 10 lần lặp $\times$ 5 fold không phải 50 quan sát độc lập — mỗi lần lặp dùng lại đúng 1.044 dòng gốc nên các fold-train giữa các lần lặp chồng lấn nhau, khiến sai số chuẩn “ngây thơ” ($\text{std}/\sqrt{n}$) bị đánh giá thấp. Công thức hiệu chỉnh, tính trên từng **fold riêng lẻ** (50 điểm số, không phải 10 điểm trung bình):

$$\text{Var}_{\text{corrected}} = \left(\frac{1}{n} + \frac{n_{\text{test}}}{n_{\text{train}}} \right) \cdot s_{\text{fold scores}}^2$$

Vì $n_{\text{test}}/n_{\text{train}} > 0$ luôn dương, phương sai hiệu chỉnh **luôn** $\geq$ phương sai ngây thơ — chỉ có thể làm CI rộng ra hoặc giữ nguyên, không bao giờ làm hẹp lại giả tạo. Đây là lý do CI hiệu chỉnh trong bảng M2.5 luôn rộng hơn CI bootstrap “ngây thơ” đúng hệ số 3,67 lần cho mọi cặp model.

3.2.3 Quyết định lại hyperparameter C=0,1 của LogReg/SVM — đã kiểm định lại, kết quả MÃU THUẦN

Nested-CV trên tập Tune (730 dòng, mục ở trên) cho thấy **không có bằng chứng ủng hộ** C=0,1: LogReg default(C=1,0) 0,689 $\rightarrow$ tuned 0,681 ($\Delta=-0,008$, CI[-0,019,+0,002], p=0,124); SVM default 0,666 $\rightarrow$ tuned 0,662 ($\Delta=-0,003$, CI[-0,014,+0,008], p=0,590) — cả 2 CI đều chạm 0, và tham số thắng ở từng outer fold còn không ổn định (LogReg đổi giữa C=0,1/0,3 tùy fold).

Để kiểm tra lại một cách độc lập, vừa chạy thêm phép so sánh **default C=1,0 vs C=0,1 bằng 10×5-fold CV chạy trực tiếp trên 314 dòng Holdout** (tập chưa từng dùng để chọn C=0,1 — hàm `holdout_check` có sẵn trong `tune_hyperparameters.py`, cờ `--holdout-csv`). Kết quả **ngược hẳn** với nested-CV trên Tune:

Model	Default (C=1,0)	Tuned (C=0,1)	Δ	95% CI	p	Kết luận
Logistic Regression	0,629	0,655	+0,026	[+0,007, +0,046]	0,008	loại 0
SVM	0,604	0,621	+0,017	[+0,001, +0,034]	0,044	loại 0

Đọc kết quả một cách trung thực — đây KHÔNG phải bằng chứng văn hồi cho C=0,1:

- Đây là 2 phép đo trên 2 mẫu **khác nhau và không chồng lấn** (nested-CV trên 730 dòng Tune vs CV trực tiếp trên 314 dòng Holdout) — không phải “train trên Tune, kiểm 1 lần trên Holdout” như cách làm ở mục xác nhận đặc trưng lexicon. Việc 2 mẫu cho 2 kết luận trái chiều nhau nhiều khả năng chỉ phản ánh **nhiều lấy mẫu** ở quy mô dữ liệu còn nhỏ (314 dòng chia 5 fold $\square$ ~63 dòng/fold), chứ chưa chắc phản ánh một hiệu ứng thật ổn định.
- Về mặt kỷ luật thống kê, chính docstring của `tune_hyperparameters.py` quy định: chỉ nên chạy bước xác nhận trên Holdout **cho ứng viên đã “vượt ngưỡng” trên tập Tune trước** — nhưng cả LogReg lẫn SVM đều **không** vượt ngưỡng đó (CI chạm 0 ở bước nested-CV). Vì vậy phép kiểm định trên Holdout lần này về bản chất là một **phân tích khám phá** (không nằm trong quy trình xác nhận đã định trước), nên kết quả có ý nghĩa thống kê ở đây cần được diễn giải **thận trọng hơn** một xác nhận chính thức.
- **Kết luận cân bằng:** bằng chứng hiện tại **mâu thuẫn giữa 2 mẫu**, không đủ để chốt theo hướng nào. Giữ nguyên C=0,1 trong code production là lựa chọn hợp lý tạm thời (không có bằng chứng đòi phải đổi ngay, và Holdout — mẫu độc lập hơn với chính quy trình chọn C — lại nghiêng nhẹ về phía ủng hộ nó), nhưng **chưa nên coi đây là đã “quyết định xong”** — cần ground truth mới, độc lập với cả Tune lẫn Holdout hiện tại, để phân xử dứt điểm.

3.3 3. Transfer Learning (PhoBERT)

3.3.1 Luồng làm việc

Lộ trình 4 bước, mỗi bước kế thừa trọng số bước trước (trừ E2 chạy song song E3 như baseline đối chứng), dùng chung `vinai/phobert-base-v2` (12 lớp, hidden 768, ~135 triệu tham số):

1. **E1 — Domain-Adaptive Pretraining (DAPT)**: pack câu (không cắt giữa câu, không trộn câu của 2 bài khác nhau) vào các khối ≤ 256 token, che ngẫu nhiên **15%** token (MLM, `mlm_probability=0,15`), huấn luyện tiếp trên 125.511 bài **đã loại hết** các bài thuộc ground truth — tránh mô hình “đọc trước” nội dung mà E2/E3 sẽ dùng để đánh giá. Siêu tham số: 3 epoch, batch size 32, learning rate $5e-5$.
2. **E2 — Frozen feature probe**: đóng băng encoder đã DAPT (`torch.no_grad()`), với bài dài hơn 256 token (~37% số bài) thì **chia thành nhiều cửa sổ ≤ 256 token theo câu** thay vì cắt cụt — mỗi cửa sổ mean-pool `last_hidden_state` theo attention mask, rồi gộp các cửa sổ bằng trung bình có trọng số theo độ dài, cho ra đúng 1 vector 768 chiều/bài. Chỉ huấn luyện một LogisticRegression multinomial nhẹ (2.307 tham số) trên vector này.
 - **Cơ chế chia cửa sổ** (`chunk_sentences_into_windows`, dùng chung cho cả E2/E3/E4): đóng gói (bin-packing) **tham lam theo thứ tự câu** — gộp lần lượt từng câu vào cửa sổ hiện tại cho tới khi câu tiếp theo làm vượt quá 254 token (256 trừ 2 vị trí cho BOS/EOS) thì mở cửa sổ mới; không có overlap giữa 2 cửa sổ liên tiếp. Trường hợp hiếm (~1/80k câu) 1 câu tự nó đã dài hơn cửa sổ thì cắt cứng theo từ (words) làm phương án dự phòng cuối. Trọng số gộp = **số từ** trong cửa sổ đó (cửa sổ dài hơn đại diện cho phần lớn hơn của bài).
3. **E3 — Fine-tune toàn phần**: khởi tạo từ checkpoint E1, thay MLM head bằng classifier head 3 lớp mới (`ignore_mismatched_sizes=True`), lan truyền ngược qua **toàn bộ mạng** bằng `WeightedLossTrainer` — `CrossEntropyLoss` với trọng số lớp $w_i = N/(K \cdot n_i)$ (N =tổng số dòng train của fold, $K=3$ lớp, n_i =số dòng lớp i) tính riêng trên từng fold-train, dùng lại đúng cơ chế chia cửa sổ + gộp trọng số của E2 (không train trên vector cố định mà train trực tiếp trên từng cửa sổ, gộp xác suất khi dự đoán). Siêu tham số: 5 epoch, batch size 16, learning rate $2e-5$, weight decay 0,01, mixed-precision bf16 trên GPU.
4. **E4 — Distillation trên 126k bài chưa nhãn**: model “trò” train **lại từ đầu** (từ checkpoint E1, không kế thừa E3) trên một tập hợp gồm 2 loại ví dụ dùng **chung một hàm loss** — soft cross-entropy trên vector xác suất:

$$\mathcal{L} = - \sum_{k=1}^3 q_k \log(\hat{p}_k)$$

với q = target (`[1,0,0]` dạng one-hot cho bài có nhãn thật; hoặc xác suất mềm $[\hat{p}_{\text{neg}}^{(E3)}, \hat{p}_{\text{neu}}^{(E3)}, \hat{p}_{\text{pos}}^{(E3)}]$ mà **chính model E3 (thầy) đã chấm sẵn từ trước** cho bài lấy mẫu **ngẫu nhiên thuần** — không có chiến lược active-learning/stratify — từ 126k bài chưa nhãn, loại hết `source_row_id` đã có trong ground truth). Vì điểm thầy đã chấm sẵn (đọc từ file), bước train này **không cần forward-pass lại qua model E3**. Phân biệt rõ **pilot** (chỉ train trên Tune 730 + mẫu chưa nhãn, đánh giá trung thực trên Holdout) và **production** (train trên toàn bộ 1.044 dòng + mẫu chưa nhãn, dùng để chấm điểm 126k bài thật, không giữ riêng phần đánh giá vì đã đánh giá xong ở bước pilot).

5. **Giao thức đánh giá** dùng chung cho E2/E3: 5-fold CV một lần trên toàn bộ 1.044 dòng, repeated 10×5-fold CV trên Tune (730 dòng), và Holdout (314 dòng) chỉ kiểm tra **đúng một lần**.

Bước	Kết quả
E1 (DAPT)	Perplexity 8,30 → 3,80
E2 (frozen probe)	Macro F1 0,748 (CV 1.044 bài), ổn định qua seed
E3 (fine-tune)	Macro F1 0,769 (CV), 0,7709±0,0024 qua 5 seed
E4 (distillation, N=20.000 bài chưa nhãn)	Macro F1 tới 0,83 (Holdout) — cải thiện lớn nhất nhánh

Đã sửa 2 vấn đề kỹ thuật quan trọng: **truncation 256 token** (~37% số bài dài hơn, đã sửa bằng chia cửa sổ + gộp trọng số thay vì cắt cụt) và bổ sung **chỉ số đo chất lượng điểm liên tục** (không chỉ macro-F1 argmax) vì bước dựng chỉ số VN-Index dùng điểm số liên tục prob_positive – prob_negative:

	Spearman (điểm liên tục vs nhãn người)		MAE (thang –1/0/1)
Phương pháp			
Lexicon (PMI)	0,435		0,509
Random Forest	0,598		0,397
PhoBERT	0,734		0,305

3.3.2 Kiểm định độ chính xác của việc triển khai

- **Chống rò rỉ ở gốc:** toàn bộ bài thuộc ground truth bị loại khỏi corpus DAPT trước khi train E1 — xác nhận bằng code, không chỉ bằng giả định — để mô hình không “thấy trước” nội dung sẽ dùng đánh giá E2/E3.
- **Một cơ chế chunk+pool dùng chung cho mọi bước:** hàm chia cửa sổ theo câu (chunk_sentences_into_windows) và gộp trọng số theo độ dài (pool_chunks_by_weight) được viết **một lần** trong model/common.py rồi import y hết vào E2, E3, E4 và cả script chấm điểm 126k bài corpus — loại bỏ rủi ro nhiều nơi cài đặt cùng một logic nhưng lệch nhau (lỗi rất dễ xảy ra khi copy-paste chunking logic ở nhiều file).
- **Một lỗi cài đặt thật đã bắt được nhờ crash, không phải nhờ số liệu sai âm thầm:** khi train distillation (E4), HF Trainer mặc định tự động loại bỏ cột không nằm trong forward signature của model (remove_unused_columns=True), khiến cột soft_label bị strip trước khi tới collator, gây KeyError. Bắt lỗi và sửa bằng cách set remove_unused_columns=False tường minh trước khi train — nếu không có crash này, model có thể đã âm thầm train sai (không đúng soft-label) mà vẫn chạy hết bình thường.
- **Một lỗi trong chính code đánh giá cũng đã bắt được:** tính Spearman/MAE cho điểm số liên tục ban đầu join theo cột id, nhưng ground_truth_combined.csv có 12 giá trị id trùng lặp khiến kết quả join ra n=1056 thay vì 1044 — sửa bằng cách join theo **vị trí** (positional alignment), xác minh lại bằng khớp chính xác cột title ở từng dòng.

- **Xác minh join được nhúng thẳng vào hàm nạp dữ liệu, không phải một bước kiểm tra rời rạc để quên chạy:** mỗi bài trong ground truth được nối với đúng dòng của nó trong corpus đã tokenize (để lấy Tokenize_content_sentences) bằng source_row_id theo vị trí; load_ground_truth() tự động so khớp lại cột title ở từng dòng sau khi nối và **raise lỗi ngay** nếu có bất kỳ dòng nào lệch — không thể vô tình chạy tiếp trên dữ liệu đã nối sai mà không biết.
- **Kiểm tra ổn định qua seed như một phép kiểm định triển khai:** train lại E3 5 lần với 5 seed weight-init khác nhau, **giữ cố định seed chia fold** để cô lập đúng một nguồn ngẫu nhiên — độ lệch chuẩn kết quả chỉ 0,0024 (~0,3% giá trị trung bình), cho thấy pipeline train không có hành vi bất định ngoài ý muốn.
- **Đối chiếu chéo 2 phương pháp tính khoảng tin cậy độc lập:** CI theo bootstrap và CI theo delta method (Wald, tính trên confusion matrix, không liên quan gì tới cách bootstrap resample) cho ra khoảng gần như trùng khít trên trường hợp có cả 2 số liệu (Holdout trước distillation: bootstrap [0,7078, 0,8028] vs delta [0,7100, 0,8062]) — xác nhận cả 2 cách tính CI đều được cài đặt đúng, không phải trùng hợp ngẫu nhiên của riêng một phương pháp.

4 So sánh 3 nhánh trên cùng Holdout (314 bài, kiểm tra một lần)

Nhánh	Macro F1	95% CI (bootstrap)
Lexicon (PMI)	0,6500	[0,595, 0,707]
Random Forest	0,7149	[0,662, 0,765]
PhoBERT E3 (trước distillation)	0,7581	[0,708, 0,803]

Paired bootstrap + McNemar: **PhoBERT và Random Forest đều tách biệt đáng tin khỏi Lexicon** (CI loại 0), nhưng **PhoBERT và Random Forest không phân biệt được với nhau** (CI chạm 0) — kết luận đúng: **TL $\square$ ML > Lexicon**, không phải “TL thắng tuyệt đối”. (Bảng này dùng bản PhoBERT trước distillation vì đây là số duy nhất chưa bị ảnh hưởng bởi việc chọn siêu tham số dựa trên chính Holdout.)

5 Câu hỏi chính: sentiment tin tức có dự báo được VN-Index không?

5.1 Kết luận: KHÔNG — null vững qua mọi kiểm định

Hồi quy kiểu Tetlock (2007) cấp ngày (Newey-West HAC 5 lag) giữa sentiment và **lợi suất bất thường** (abnormal return) của VN-Index cho kết quả **null ở cả 6 biến thể đo sentiment** (Lexicon PMI/Intensity/PCA-PMI/PCA-Intensity, Random Forest, PhoBERT). Vì ban đầu nghi ngờ null này là do **sai số đo lường** (attenuation bias — phương pháp kém chính xác kéo hệ số về 0), đã kiểm chứng bằng nhiều hướng độc lập, **tất cả đều hội tụ về cùng kết luận null là thật**:

Kiểm định	Kết quả
Gradient test (hệ số có mạnh dần theo độ chính xác classifier không)	Không — PhoBERT (đo tốt nhất, Spearman 0,73) cho hệ số nhỏ nhất , p-value lớn nhất (0,77) trong 6 phương pháp — ngược hẳn dự đoán của attenuation bias
2SLS (Lexicon làm công cụ, sửa trực tiếp attenuation bias)	Vẫn null ($p=0,95-0,99$); artifact “lag4” từng lặp lại ở hầu hết hồi quy OLS mất ý nghĩa dưới 2SLS
Hiệu chỉnh xác suất (isotonic calibration)	Cải thiện Brier/Spearman thật, không đổi kết luận
Ổn định theo seed (3-5 seed)	Cực ổn định
Đảo chiều nhân quả (return trễ $\rightarrow$ sentiment hiện tại, kiểm tra “báo đăng chậm hơn giá”)	Null ở lag1 tại cả 6 phương pháp
Tương quan cùng ngày (kiểm tra “thị trường hấp thụ tin trong ngày”)	Gần như bằng 0 ở cả 6 phương pháp
Power / Minimum Detectable Effect	MDE chỉ chiếm 5–10% độ lệch chuẩn lợi suất 1 ngày — loại trừ được cả hiệu ứng nhỏ, không phải do thiếu power

Kết luận null còn giữ nguyên sau khi sửa 1 lỗi look-ahead nhẹ phát hiện trong biến kiểm soát biến động (volatility_20d trước đó vô tình dùng một phần return đang dự báo) và với model tốt nhất từng có (PhoBERT distillation).

Đọc kết quả đúng: đây không phải là “dự án thất bại” — null với đầy đủ kiểm chứng power/robustness **là một kết quả khoa học hợp lệ**: tin tức về một công ty đơn lẻ (bản chất >99% corpus) không có cơ chế rõ ràng để dịch chuyển cả một chỉ số thị trường ~400 mã, trong khung thời gian và độ trễ đã kiểm tra.

6 Phát hiện phụ đáng chú ý: khối lượng giao dịch khối nội

Trong lúc kiểm định lợi suất null, phát hiện: sentiment (Lexicon PMI/Intensity) có tương quan có ý nghĩa với **khối lượng giao dịch khối NỘI địa** ngày hôm sau, nhưng **không** với khối ngoại — sau khi kiểm soát cả year fixed-effect lẫn số lượng bài/ngày (article_count, để tách “hướng tin” khỏi “lượng tin”/attention). Phát hiện này:

- Khớp với văn liệu học thuật về **attention của nhà đầu tư cá nhân**: Barber & Odean (2008) — nhà đầu tư cá nhân mua theo cổ phiếu “được chú ý”; Da, Engelberg & Gao (2011) — attention đo bằng Google Search Volume dự báo giá ngắn hạn; và một nghiên cứu xuyên 8 thị trường mới nổi (Debata et al.) kết luận trực tiếp “*domestic investors drive sentiment trades, whereas foreign investors are not biased*”.
- Vượt qua **permutation test** nghiêm ngặt (1.999 lần xoay vòng ngẫu nhiên chuỗi sentiment): empirical p-value 0,002 (PMI) và 0,0035 (Intensity) — không phải artifact thống kê.
- Không có bằng chứng nhân quả ngược (sentiment không bị dự báo bởi lợi suất trễ ở Lexicon/Random Forest).

- Nhưng **KHÔNG nhất quán khi kiểm tra đủ 6 phương pháp**: chỉ 2/6 (Lexicon PMI, Intensity) cho khối nội có ý nghĩa; PhoBERT (phương pháp đo tốt nhất) lại cho pattern **ngược lại** (khối ngoại có ý nghĩa, khối nội không); Random Forest và 2 biến thể PCA đều null cả 2 phía.

Kết luận cân bằng: tín hiệu này *có thật* ở Lexicon (không phải nhiễu ngẫu nhiên, đã kiểm chứng bằng permutation test) và có nền tảng lý thuyết vững, nhưng *chưa nhất quán đủ giữa các phương pháp đo* để khẳng định chắc chắn. Đây là hướng **tiềm năng nhất** nếu dự án muốn phát triển thành một bài báo (mentor coi là “trục chính” thay vì dự báo VN-Index).

6.0.1 Vì sao PhoBERT và Lexicon cho kết quả khác nhau — đã điều tra trực tiếp trên dữ liệu

So sánh điểm `net_sentiment_score` cấp bài giữa Lexicon PMI và PhoBERT trên toàn bộ 126.576 bài (cùng thứ tự dòng, đã xác minh khớp `title` 100%):

- **Tương quan chỉ ở mức trung bình**: Pearson $r=0,406$, Spearman $\rho=0,454$ — 2 phương pháp thật sự đo 2 tín hiệu khác nhau đáng kể, không phải cùng 1 tín hiệu có nhiễu khác nhau.
- **37% số bài Lexicon chấm “không khớp từ nào”** (`positive_score=negative_score=0`) — nhưng trong nhóm này, PhoBERT vẫn đưa ra quan điểm rõ ràng ở 24% số bài (15,1% positive + 8,9% negative), và ngay cả phần PhoBERT gọi neutral, điểm liên tục trung bình `|net_sentiment_score|` vẫn là 0,23 (so với 0,45 toàn corpus) — PhoBERT đọc được sentiment từ nhiều bài mà Lexicon hoàn toàn không có tín hiệu.
- **Nguyên nhân cụ thể, truy được tận gốc dictionary**: xem các bài có 2 điểm số **trái dấu mạnh nhất**, ví dụ Lexicon rất tích cực (+) nhưng PhoBERT rất tiêu cực (-):

Tiêu đề	Lexicon	PhoBERT
“VNG: Quý 2 lỗ 867 triệu đồng”	+0,055	-0,947
“SD7: Lãi trước thuế công ty mẹ giảm 4,3 tỷ...”	+0,048	-0,948
“TDH: Sau soát xét, bất ngờ chuyển từ lãi sang lỗ hơn 7 tỷ đồng”	+0,041	-0,942
“IJC: Quý 1 thực hiện 7% kế hoạch lợi nhuận”	+0,040	-0,943

Cả 4 tiêu đề đều rõ ràng là tin **xấu** (lỗ, lãi giảm, chuyển từ lãi sang lỗ, chỉ đạt 7% kế hoạch) mà PhoBERT nhận diện đúng, nhưng Lexicon lại chấm dương. Tra lại `weighted_dictionary.csv` cho thấy nguyên nhân là **giới hạn cấu trúc của cách khớp cụm từ cố định**: từ điển không có từ “tăng”/“giảm” trần trụi (chỉ có các cụm cố định như “lợi_nhuận sụt_giảm”, “giảm mạnh” được gán negative), nên khi câu viết theo cách khác — “lãi ... giảm 4,3 tỷ” hay “lãi ... chuyển sang lỗ” (từ không nằm liền kề đúng như

cụm đã định nghĩa) — không cụm âm nào khớp được, trong khi từ dương (lãi, lợi_nhuận, kế_hoạch...) vẫn xuất hiện và kéo điểm lên dương. Ở chiều ngược lại (Lexicon rất tiêu cực nhưng PhoBERT rất tích cực), ví dụ rõ nhất: “*DVN tăng gần 50% lãi ròng quý 3 nhờ tiết giảm chi phí*” (tin tốt — cắt giảm chi phí giúp tăng lãi) bị Lexicon chấm âm (−0,037) vì “tiết_giảm” được gán cứng vào nhóm negative trong từ điển — không phân biệt được “giảm cái gì”: giảm **chi phí** (tốt) hoàn toàn khác giảm **lợi nhuận** (xấu), nhưng từ điển chỉ nhìn thấy “giảm” và luôn chấm âm.

- **Cơ chế gốc rễ:** Lexicon là mô hình “túi cụm từ” (đếm khớp cụm cố định, cộng dồn theo trọng số, chuẩn hoá theo độ dài bài) — không có khả năng hiểu **quan hệ giữa từ chỉ hướng biến động (tăng/giảm) và đối tượng của nó** (lợi nhuận, doanh thu, chi phí, giá vốn — mỗi đối tượng đảo ngược ý nghĩa của “tăng”/“giảm”), và một bài dài có thể bị các cụm dương/âm rải rác ở các đoạn khác nhau “pha loãng” lẫn nhau. PhoBERT, học end-to-end từ 1.044 nhãn người (qua distillation lan ra 126k bài), nắm được quan hệ ngữ cảnh này mà không cần liệt kê từng trường hợp.

Chưa kết luận được (giới hạn của phần điều tra này): việc trên giải thích tốt tại sao **2 chỉ số sentiment tự nó khác nhau** — nhưng chưa chứng minh được đây chính là lý do khiến khối NỘI phản ứng với Lexicon còn khối NGOẠI phản ứng với PhoBERT (đòi hỏi dữ liệu hành vi nhà đầu tư ở cấp vi mô mà dự án chưa có). Đây vẫn là một giả thuyết hợp lý, chưa phải bằng chứng nhân quả.

7 Kết luận chung toàn dự án

1. **PhoBERT (Transfer Learning) là phương pháp phân loại sentiment tốt nhất** trong 3 nhánh, cả về macro-F1 (0,77–0,83 sau distillation) lẫn chất lượng điểm số liên tục (Spearman 0,73) — nhưng khoảng cách với Random Forest chưa tách biệt đáng tin ở cỡ mẫu Holdout hiện có.
2. **Distillation trên 126k bài chưa nhãn** là cải thiện kỹ thuật hiệu quả nhất được thử trong toàn dự án — vượt xa mọi nỗ lực tối ưu kiến trúc/hyperparameter riêng lẻ.
3. **Sentiment tin tức cổ phiếu đơn lẻ không dự báo được lợi suất VN-Index** — kết luận null này đã được kiểm chứng nghiêm ngặt (gradient test, 2SLS, power/MDE, nhiều kiểm định độc lập khác) và giữ vững qua mọi phiên bản pipeline và mọi phương pháp đo lường, kể cả phương pháp tốt nhất.
4. **Tín hiệu ở khối lượng giao dịch khối nội** là phát hiện đáng chú ý nhất, có nền tảng lý thuyết và vượt qua permutation test, nhưng chưa đủ nhất quán giữa các phương pháp đo để công bố chắc chắn — hướng nghiên cứu tiếp theo hứa hẹn nhất của dự án.
5. **Chất lượng ground truth (1.044 bài) đã đạt trần lý thuyết hợp lý** (~0,71–0,74 theo đo lường liên chủ quan) — phần lớn “khoảng trống” còn lại của model nằm ở độ mơ hồ tự nhiên của nhãn, không phải hạn chế của mô hình.

7.1 Việc còn mở

- So sánh 3 nhánh trên tập nhãn **hoàn toàn mới** (chưa từng dùng cho bất kỳ quyết định nào) — Holdout hiện tại đã bị dùng nhiều lần cho các câu hỏi khác nhau.

- Đào sâu vì sao PhoBERT và Lexicon cho kết quả khối nội/khối ngoại trái ngược nhau.
- Cân nhắc chiến lược chọn mẫu thông minh hơn (không ngẫu nhiên thuần) cho distillation.
- Hyperparameter $C=0,1$ của LogReg/SVM ở nhánh Traditional ML: đã kiểm định lại trên Holdout, nhưng kết quả **mâu thuẫn** với nested-CV trên Tune (xem mục Traditional ML) — cần ground truth mới, độc lập với cả Tune/Holdout hiện tại, để phân xử dứt điểm thay vì chỉ dựa vào 2 mẫu đã dùng nhiều lần.